

FOSS Club

Roles Overview

■ What they'll do

✓ Requirements

→ Real-world work

■ Web Frontend developer

Builds for browsers — responsive, fast, and pixel-perfect

■ WHAT THEY'LL DO

Build UIs	React/Next.js pages, reusable component design systems
Responsive Design	Tailwind, Bootstrap, CSS Grid/Flexbox — works on every screen
Performance & SEO	Core Web Vitals, lazy loading, meta tags, Lighthouse audits
AI-Assisted Dev	Google AI Studio, Stitch for rapid UI generation & prototyping

✓ REQUIREMENTS

- ✓ HTML, CSS, JavaScript (ES6+) — solid fundamentals
- ✓ React or Next.js — component-based development
- ✓ Responsive design tools: Tailwind CSS / Bootstrap
- ✓ Component design principles & reusable UI patterns
- ✓ Git & GitHub pull-request workflow
- ✓ Basic exposure to Google AI Studio or Stitch (bonus)

→ REAL-WORLD PROJECT WORK

→ Club website, product landing pages & dashboards

→ Component library shared across all club projects

→ AI-assisted rapid UI prototyping with Stitch / AI Studio

React

Next.js

TypeScript

Tailwind CSS

Stitch

Google AI
Studio

Vercel

■ Figma literacy is expected — work closely with UI/UX before writing a single line of CSS.

■ Mobile Frontend developer

Builds native-quality apps for Android & iOS

■ WHAT THEY'LL DO

Cross-platform Apps	Flutter/React Native — one codebase, two stores
Native Development	Swift (iOS), Kotlin (Android), Rust for performance-critical modules
Routing & Navigation	Expo Router, deep linking, tab & stack navigators — must be solid
Native APIs	Camera, GPS, push notifications, Bluetooth, biometrics

✓ REQUIREMENTS

- ✓ Flutter/Dart OR React Native/JavaScript — pick one, learn both over time
- ✓ Expo & Expo Router — file-based routing, similar to Next.js
- ✓ Routing concepts: stack, tab, drawer, deep links — must be comfortable
- ✓ Basic Swift (iOS) or Kotlin (Android) for native modules
- ✓ Rust knowledge for mobile (optional but valued for performance)
- ✓ App store submission process (Play Store & App Store)

→ REAL-WORLD PROJECT WORK

→ Android/iOS companion apps for every major web project

→ Campus utility app distributed via Play Store & F-Droid

→ Performance-critical features using native Swift/Kotlin/Rust modules

Flutter

Dart

React
Native

Expo
Router

Swift

Kotlin

Rust

Android
Studio

Xcode

■ *Must understand routing deeply — Expo Router is our standard. Navigation bugs are user-facing bugs.*

■ UI/UX Design developer

Shapes how every product looks, feels, and flows

■ WHAT THEY'LL DO

Figma — Core Skill	Wireframes, auto-layout, components, variants, prototypes — Figma is non-negotiable
UML Diagrams	Use-case, activity, sequence diagrams before any feature is built
System Diagrams	Eraser.io for architecture diagrams, flowcharts, and collaborative diagramming
Design System	Colours, typography, spacing tokens — one system used across all projects

✓ REQUIREMENTS

- ✓ Figma — 90%+ of design work happens here. Non-negotiable baseline skill
- ✓ UML: use-case, activity & sequence diagrams — design before you build
- ✓ Eraser.io — collaborative system & architecture diagramming
- ✓ FigJam / Miro — brainstorming, user flows, affinity mapping
- ✓ Basic HTML/CSS — to understand what's feasible to build
- ✓ Accessibility: WCAG 2.1 contrast, sizing, and focus states

→ REAL-WORLD PROJECT WORK

→ Figma design system adopted by all frontend teams

→ UML diagrams for every new feature before dev starts

→ Eraser.io architecture diagrams for technical documentation

Figma

FigJam

Eraser.io

Penpot (OSS)

Canva

Storybook

■ Every feature ships a Figma prototype first. No prototype → no development. Figma is the single source of truth.

■ ■ Backend Engineering developer

The brain and engine behind every product feature

■ WHAT THEY'LL DO

API Design — Must	REST & GraphQL design, versioning, OpenAPI/Swagger docs — core skill
UML & Architecture	Sequence, class, ER diagrams before writing code
Microservices & Kafka	Event-driven architecture, Kafka topics, producers/consumers
Multi-language	Node.js/Express, Python/Django/FastAPI, Java/Spring Boot, Go

✓ REQUIREMENTS

- ✓ API Design: REST principles, HTTP methods, status codes, OpenAPI — must know
- ✓ UML: sequence & class diagrams — design the system before you build it
- ✓ Node.js + Express OR Python + Django/FastAPI — at least one, ideally two
- ✓ Java + Spring Boot — for enterprise & startup-grade backend work
- ✓ Golang — for high-performance services and microservices
- ✓ Apache Kafka — event streaming, message queues, async communication
- ✓ Auth: JWT, OAuth2, session management, role-based access control

→ REAL-WORLD PROJECT WORK

→ Design & maintain the API layer for all web and mobile apps

→ Build Kafka-powered event streams for real-time features

→ Spring Boot microservices for startup-grade production systems

Node.js

Express

Django

FastAPI

Java

Spring
Boot

Golang

Kafka

Docker

Postman

■ API design is a must-have, not a nice-to-have. Every endpoint must be documented in Swagger before merging.

Database Engineering developer

Keeper of all data — structured, unstructured, and vectorised

WHAT THEY'LL DO

Schema & ER Diagrams	Design ER diagrams before touching a database — mandatory practice
SQL & NoSQL	PostgreSQL, MySQL, MongoDB, Redis, Firebase Realtime & Firestore
ORM Mastery	Prisma (Node), SQLAlchemy (Python), Hibernate (Java) — ORM is the bridge
Vector & AI DBs	pgvector for PostgreSQL, Pinecone, Chroma — AI-ready data storage

✓ REQUIREMENTS

- ✓ SQL: joins, aggregations, window functions, EXPLAIN ANALYSE
- ✓ ER Diagrams: design before you create any schema — non-negotiable
- ✓ ORM tools: Prisma / SQLAlchemy / Sequelize — abstract the query layer
- ✓ Firebase: Realtime Database & Firestore for rapid prototyping
- ✓ pgvector: vector extension on PostgreSQL for AI/semantic search
- ✓ NoSQL patterns: embedded vs referenced, indexing strategies
- ✓ Backup, migrations & zero-downtime deployments

→ REAL-WORLD PROJECT WORK

→ Shared database schema with ER diagrams for all club projects

→ Firebase backend for rapid MVP prototyping

→ pgvector-powered semantic search for the AI/ML team's RAG systems

PostgreSQL	pgvector	MongoDB	Redis	Firebase	Prisma	SQLAlchemy	Hibernate	TablePlus
------------	----------	---------	-------	----------	--------	------------	-----------	-----------

■ Every new database must have an ER diagram committed to the repo. No diagram = no merge.

■ DevOps Engineering developer

Ships code to users — automatically, reliably, at scale

■ WHAT THEY'LL DO

CI/CD Pipelines	GitHub Actions & Jenkins — auto build, test, deploy on every push
Containers & K8s	Docker for packaging, Kubernetes for orchestration, Helm for packaging K8s apps
Cloud Infra	AWS, GCP, Vercel — hosting, domains, SSL, auto-scaling
Monitoring	Grafana, Sentry, Datadog — uptime, errors, and performance dashboards

✓ REQUIREMENTS

- ✓ Linux command line & Bash scripting — daily tool
- ✓ Docker: build, push, compose, multi-stage builds
- ✓ Kubernetes: pods, services, deployments, namespaces
- ✓ Helm: package and manage Kubernetes applications
- ✓ GitHub Actions & Jenkins: pipeline as code, secrets, matrix builds
- ✓ Cloud platforms: AWS or GCP basics (EC2, S3, Cloud Run)
- ✓ Networking: DNS, HTTP/HTTPS, TLS, load balancers

→ REAL-WORLD PROJECT WORK

→ Every PR gets a live preview URL
— zero manual steps

→ Kubernetes cluster managing all
production club services

→ Helm charts for one-command
deployment of any club project

GitHub
Actions

Jenkins

Docker

Kubernetes

Helm

AWS/GCP

Grafana

Sentry

Bash

■ *Helm is our standard for K8s deployments. If you're deploying to Kubernetes without a Helm chart, do it again.*

■ Cybersecurity & Testing developer

Defender and quality gatekeeper of every product

■ WHAT THEY'LL DO

Cryptography Fundamentals	SHA-256, salting, hashing, bcrypt — understand, implement, audit
Auth & Secure Systems	JWT, OAuth2, Auth0, session security, RBAC
Vulnerability Scanning	OWASP ZAP, Burp Suite, Snyk, dependency audits on every PR
Testing Frameworks	Jest/Cypress (JS), JUnit/Mockito (Java), PyTest (Python)

✓ REQUIREMENTS

- ✓ Cryptographic algorithms: SHA-256, MD5 (why not to use it), salting & hashing passwords
- ✓ Hashing vs encryption vs encoding — know the difference cold
- ✓ Testing: Jest + Cypress (frontend), JUnit + Mockito (Java backend), PyTest (Python)
- ✓ OWASP Top 10 — understand each vulnerability and its fix
- ✓ Auth security: JWT expiry, refresh tokens, OAuth flows
- ✓ Penetration testing basics: scanning, recon, exploit awareness

→ REAL-WORLD PROJECT WORK

→ Security audit + pen-test report before every major release

→ Full test suite: unit (JUnit/Jest) + integration + e2e (Cypress)

→ Dependency scanning via Snyk/Dependabot on every pull request

Jest

Cypress

JUnit

Mockito

PyTest

OWASP
ZAP

Burp
Suite

Snyk

bcrypt

JWT/OAuth

■ Salting and hashing are Day 1 knowledge. Storing plain-text passwords or weak hashes is an automatic reject in code review.

■ AI/ML & Prompt Engineering developer

Makes every product intelligent and autonomous

■ WHAT THEY'LL DO

LLM Integration	Claude, GPT, Groq APIs — plug AI into every product layer
Local AI & OSS Models	Ollama for local LLMs, Hugging Face for open-source model hosting
RAG & Vector DBs	LlamaIndex, LangChain, DSPy — embeddings, Qdrant, pgvector, Chroma
AI Tooling & Agents	Claude Code, Cursor, Windsurf — must work efficiently with AI dev tools

✓ REQUIREMENTS

- ✓ LLM APIs: Claude (Anthropic), Groq, OpenAI — know rate limits, pricing, context windows
- ✓ Ollama — run LLMs locally, model pulling, Modelfile customisation
- ✓ Hugging Face — model hub, Transformers library, Inference API
- ✓ LlamaIndex & DSPy — faster, structured approach vs vanilla LangChain
- ✓ Embeddings & Vector DBs: Qdrant, Chroma, pgvector — semantic search & RAG pipelines
- ✓ Multi-model orchestration — route tasks to the right model efficiently
- ✓ AI coding tools: Claude Code, Cursor, Windsurf — must be fluent and fast

→ REAL-WORLD PROJECT WORK

→ RAG chatbot over club docs using LlamaIndex + Qdrant

→ Local AI features using Ollama — no API cost, runs on-device

→ DSPy-optimised prompt pipelines for consistent, structured outputs

Claude
API

Groq

Ollama

Hugging
Face

LlamaInde
x

DSPy

LangChai
n

Qdrant

pgvector

Cursor

■ *Must be comfortable with AI coding tools (Claude Code, Cursor) — they are multipliers, not shortcuts. Multi-model orchestration is a senior expectation.*

■ Community, Operations & Social Manager

Keeps the team moving, the repo alive, and the brand visible

■ WHAT THEY'LL DO		✓ REQUIREMENTS
Scrum Master	Sprint planning, standups, retros — Agile ceremonies run weekly	✓ Agile/Scrum basics: sprint planning, backlog grooming, retrospectives
OSS Community	CONTRIBUTING.md, good-first-issue labels, issue triage, Hacktoberfest/GSoC	✓ GitHub: issues, project boards, milestones, labels — daily use
Social Media	LinkedIn page, Instagram, Twitter/X — post consistently, grow the brand	✓ Social media management: LinkedIn & Instagram content creation
Documentation	READMEs, wikis, changelogs — if it's not written, it doesn't exist	✓ Strong written communication — posts, docs, and announcements
		✓ Canva or Figma for social graphics and event posters
		✓ Event planning: hackathons, workshops, speaker sessions

→ REAL-WORLD PROJECT WORK		
→ Run all sprints across every active project (bi-weekly cadence)	→ Manage GitHub org: issues triaged within 48 hrs, docs always up-to-date	→ Grow LinkedIn + Instagram — showcase member work, releases & events

GitHub Projects	Linear	Notion	Discord	Canva	LinkedIn	Instagram	Luma
-----------------	--------	--------	---------	-------	----------	-----------	------

■ This role is glue. Without it, great code sits unseen and sprints drift. Social presence = club credibility in the real world.

■ Research & Development (R&D) Team

Explores what's next — papers, experiments, and new tech

■ WHAT THEY'LL DO

Tech Research	Evaluate new frameworks, libraries, and tools before the team adopts them
Paper Reading	Read & summarise arXiv / ACM / IEEE papers relevant to club projects
Proof of Concepts	Build small PoC projects to validate new tech before full adoption
Knowledge Sharing	Monthly tech talks, internal blog posts, and documentation of findings

✓ REQUIREMENTS

- ✓ Strong curiosity — you read docs and papers for fun
- ✓ Ability to evaluate trade-offs: performance, maintainability, learning curve
- ✓ Basic competence across the stack (not a specialist, but a generalist)
- ✓ Technical writing — clear, concise research summaries
- ✓ Comfort with uncertainty — R&D; means exploring the unknown
- ✓ Experience with at least one domain: AI/ML, systems, web, security, etc.

→ REAL-WORLD PROJECT WORK

→ Evaluate & recommend new tools before the team wastes time on wrong choices

→ Research emerging AI techniques (fine-tuning, agents) for club AI projects

→ Monthly internal tech-talk: 'What's new & what should we adopt?'

arXiv

Google Scholar

Notion (research notes)

GitHub

Jupyter Notebooks

Any relevant new tech

■ R&D; outputs feed every other team. A great PoC from R&D; can become a production feature in 4 weeks. Quality of thinking > speed of shipping.